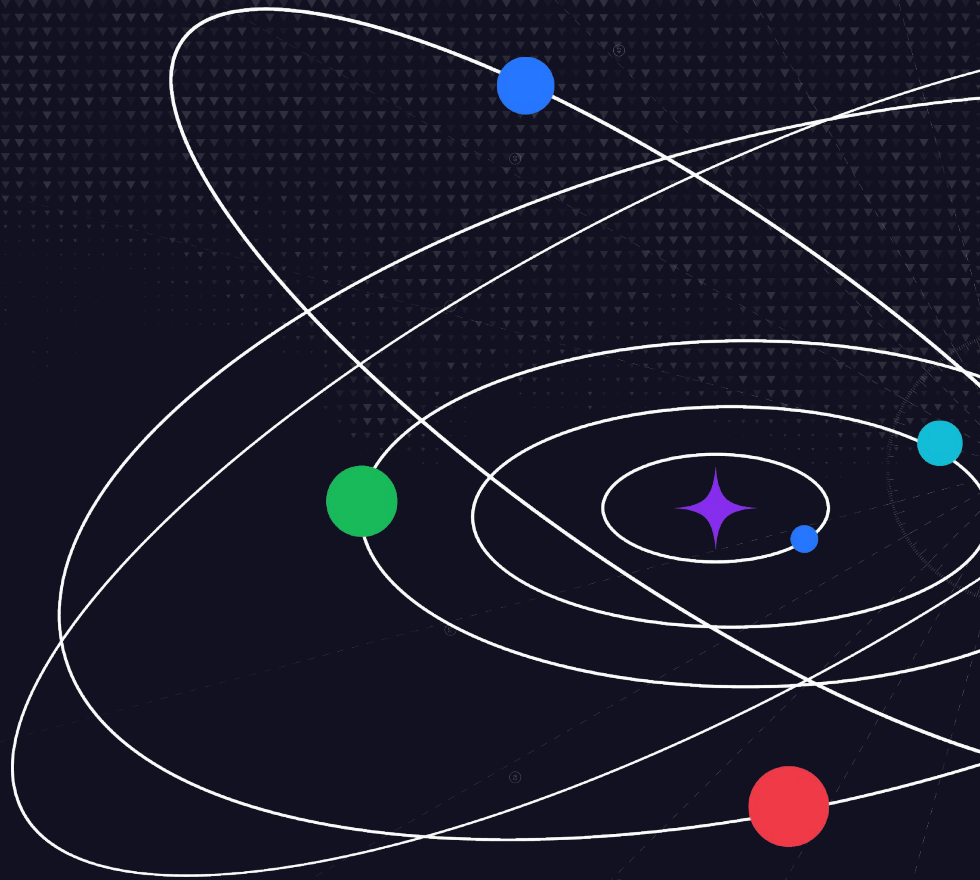


Toward a Polyglot Airflow

Tzu-ping Chung
Airflow Summit 2026

ASTRONOMER



***THIS IS NOT A
LANGUAGE SDK
WALKTHROUGH***



From JAR to DAG:

Running Java Tasks
Natively with the Airflow
Java-SDK

Zhe You Liu
Apache Airflow Committer
Software Engineer @ Astronomer



Outlook

- How Airflow runs Python
- Dag → structure
- Where is the body?
- The environment to run against

About me

- Astronomer
 - Airflow PMC member
 - Python packaging
 - Community organiser
-
- Timetables
 - Dynamic task mapping
 - Assets
 - Java SDK



```
from airflow.sdk import task
```

```
@task.stub(queue="go")
```

```
def my_go_task():
```

```
    ...
```

```
my_go_task()
```

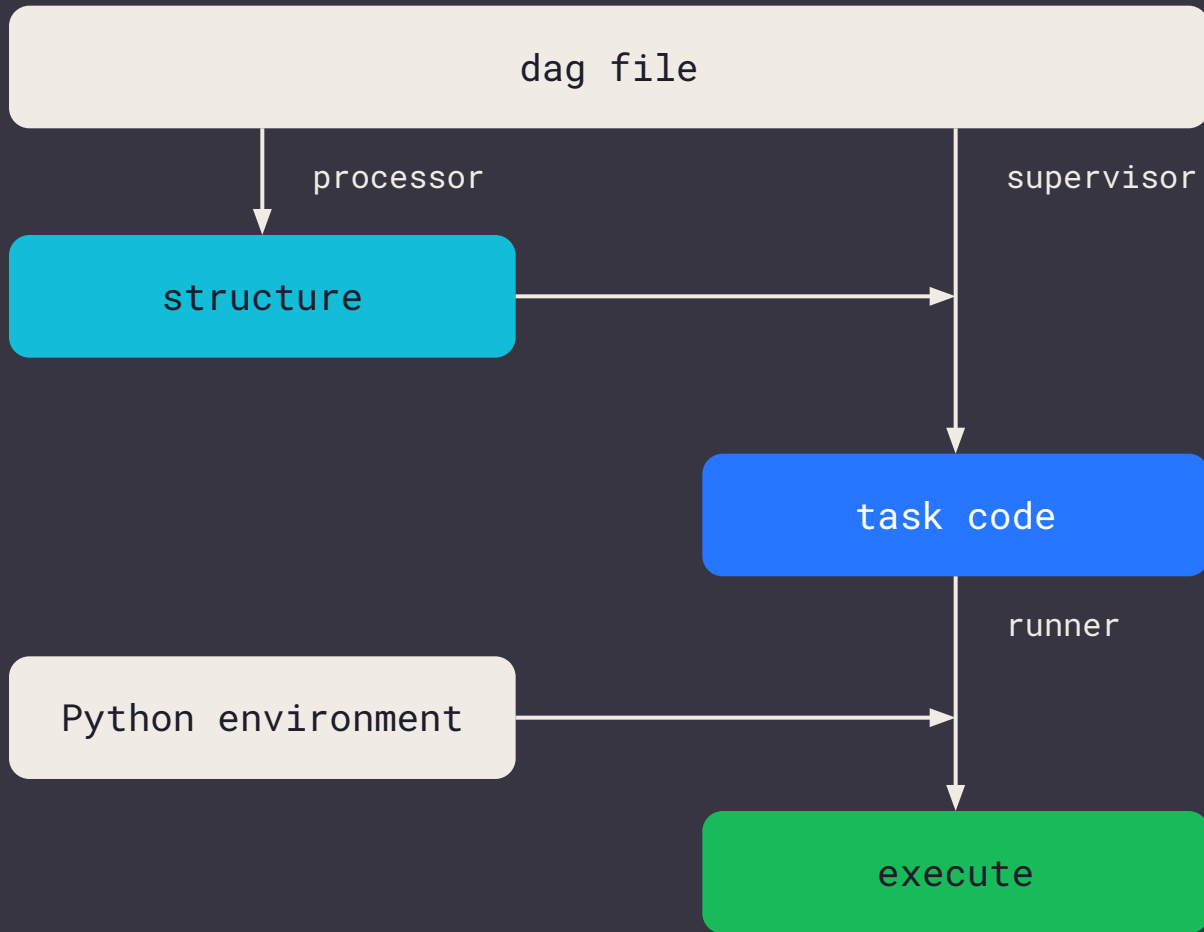
```
func (b *bundle) RegisterDags(  
    dagbag v1.Registry  
) error {  
    d := dagbag.AddDag(...)   
    d.AddTask(transform)  
    return nil  
}
```

```
func transform(  
    ctx sdk.TIRunContext,  
    client sdk.Client,  
    logger *slog.Logger  
) (any, error) {  
    ti := ctx.TaskInstance()  
    ...  
    return ...  
}
```

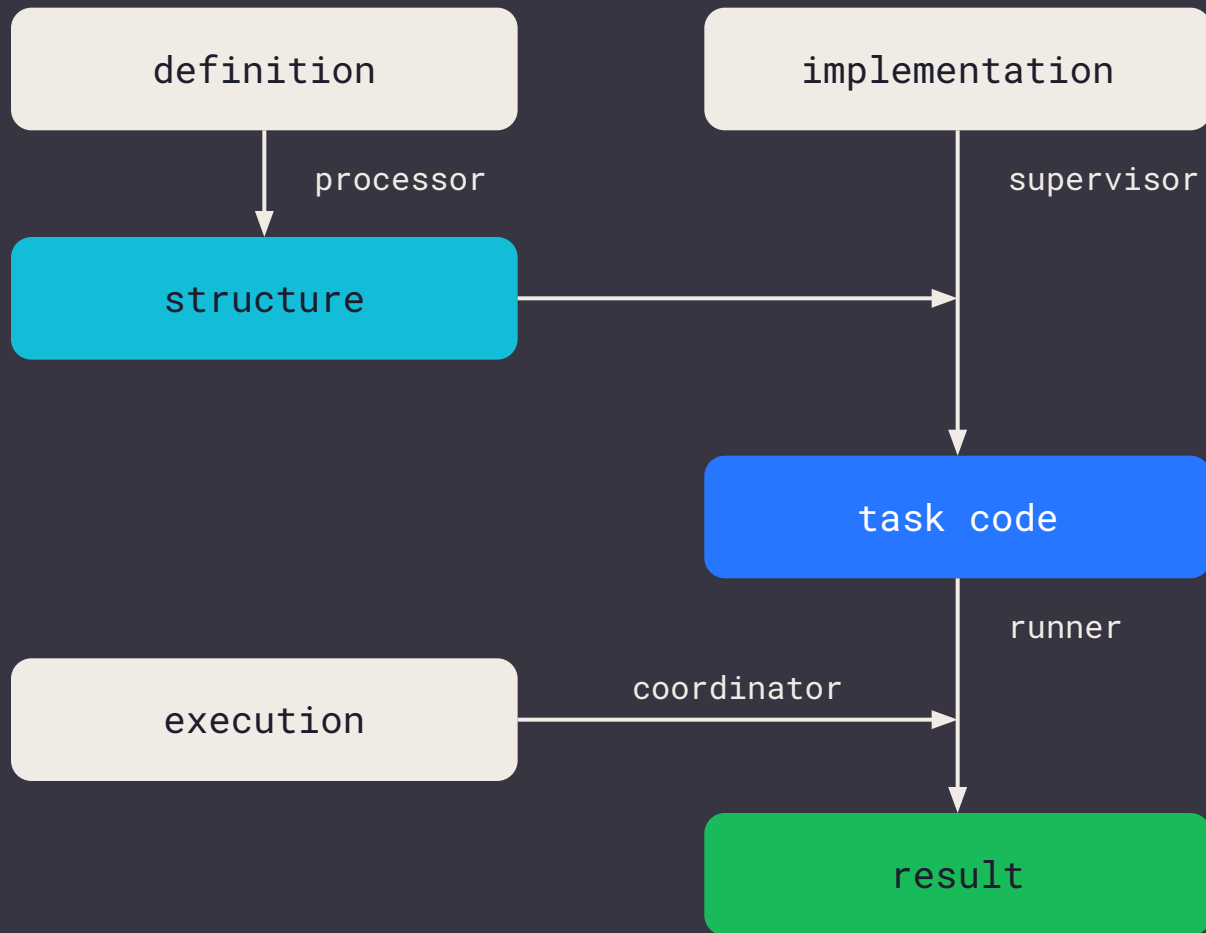
Airflow Polyglotism

ASTRONOMER

Life of a Python task



Language-agnostic



Language-agnostic Design

2

Definition

- The “shape”
- Serialized DAG
- User-facing DSLs

1

Implementation

- What a task does
- Inputs and outputs
- Programming language

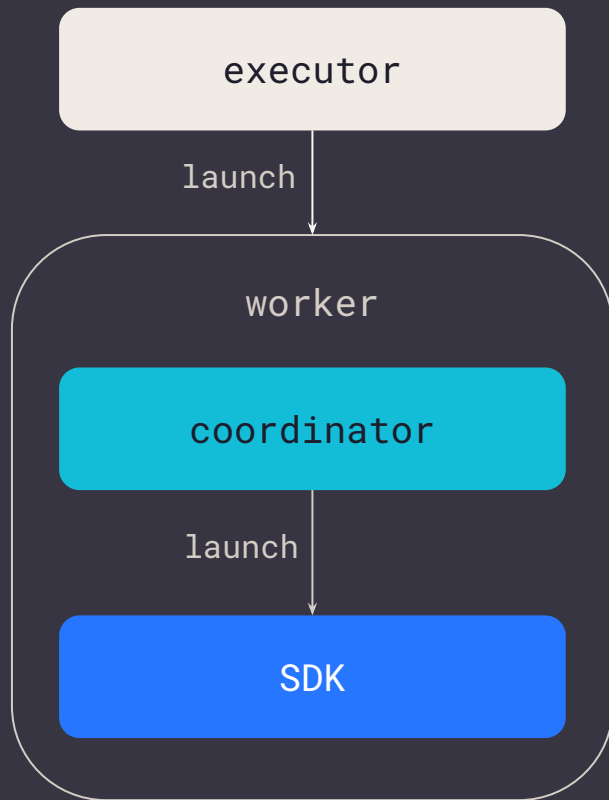
3

Execution

- Code unit
- Target platform
- Dependencies

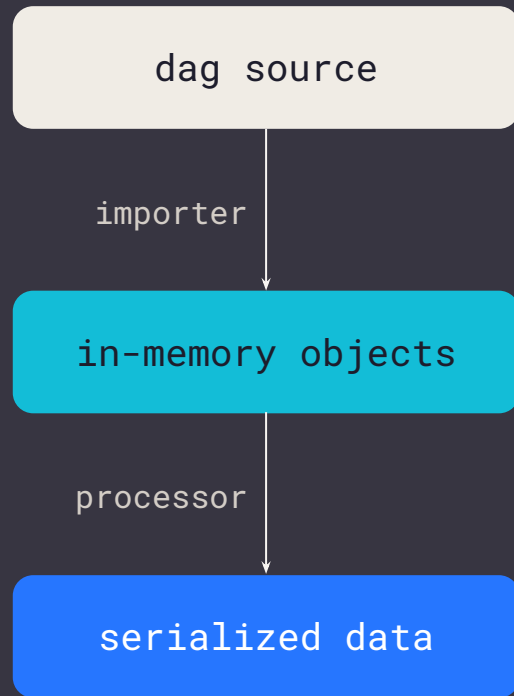
Implementation (what a task does)

- AIP-108
- Airflow 3.3+
- Currently **Java** and **Go**
- **TypeScript** soon™



Definition (chain tasks into dag)

- AIP-85
- Planned Airflow 3.4+
- What is a source?



Dag Sources in AIP-85

Static files

- JSON
- YAML
- “Safe” Python?

Executables

- Python
- Compiled binary
- *Target identification*

Exotic

- Database
- Remote objects
- *Entirely different?*

*What does it actually take
to execute a block of code?*

Execution

Airflow's current execution model

- Assumes a Python interpreter
- Uses the same environment as Airflow's
- *Just find me those packages somehow*
- Oh wait, **Kubernetes Executor**...

Why not...?

PythonVirtualEnvOperator

- Management
- Extra process
- Rich objects

ExternalPythonOperator

- Where is Python?
- Extra process
- Rich objects

KubernetesPodOperator

- What image?
- Startup time
- Rich objects

What Can a Dag Bundle Do?

- A *bundle* of resources tasks need
- User code
- Third-party packages
- Runtime? (if portable)
- How does Airflow know how to wire things up?

Let's try to make this work

- Need an interpreter?
- Need a *runtime*?
- Where do we get dependencies?
- What if the dependencies are not *managed*?
- What about *your* own libraries?

Kubernetes Executor

- Best for *you*
- Which image?
- Different runtimes, different images?
- Image management
- Combined images??

What should Airflow hide? What should you control?

```
from airflow.sdk import task

@task.stub(queue="go")
def my_go_task():
    ...

my_go_task()
```

```
func transform(
    ctx sdk.TIRunContext,
    client sdk.Client,
    logger *slog.Logger
) (any, error) {
    ti := ctx.TaskInstance()
    ...
    return ...
}
```

I don't have all the answers

- Dag Bundles should be a direction
- What should (not) be “automatic”?
- What is missing?
- Want a new SDK?
- ***Python tasks benefit too!*** (and Bash etc.)

Questions?

